

GOUTHAM KISHORE KRISHNAMOORTHY

Senior Java Backend Developer

+1 (682) 334-3001 | gouthamkishore2711@gmail.com

PROFESSIONAL SUMMARY

- **Java Backend Developer** with **7+ years** of experience designing and building scalable, high-throughput distributed systems and enterprise backend services using **Java**, **Spring Boot**, and **Microservices Architecture** across financial services and enterprise SaaS domains.
- Deep hands-on expertise in the **Spring ecosystem**, including Spring Boot, Spring MVC, Spring IOC, Spring Security, Spring Data JPA, Spring Batch, and Spring AOP, applied to production-grade backend systems handling real-time data and high-volume transactional workloads.
- Strong background in **event-driven architecture** using **Apache Kafka**, covering producer and consumer design, retry mechanisms, DLQ workflows, topic partitioning strategies, and consumer group offset management for reliable, fault-tolerant asynchronous messaging between microservices.
- Experienced in designing **RESTful APIs** with proper versioning strategies, pagination and filtering patterns, standardized error handling, and rate limiting, built to serve high-traffic frontend and downstream service consumers reliably.
- Solid expertise in **Hibernate** and **JPA** internals, covering entity lifecycle management, lazy vs. eager loading trade-offs, N+1 query detection and resolution, batch fetching, second-level cache integration, and optimized native query execution for large-scale relational datasets.
- Strong database engineering background across **PostgreSQL** and **MongoDB**, including relational schema design, index strategy (B-tree, composite, partial), query plan analysis and tuning, connection pool configuration, and MongoDB aggregation pipeline design for analytics-style workloads.
- Experienced with **Redis** for caching and beyond: TTL-based expiration, cache-aside and write-through patterns, distributed locking, and session state management to reduce database pressure and improve API latency in production systems.
- Deep understanding of **Java concurrency** and multithreading, including thread pools (ExecutorService, ForkJoinPool), synchronization primitives, CompletableFuture-based async flows, and non-blocking I/O patterns for building high-throughput, resource-efficient backend services.
- Skilled in applying backend design patterns in practice: **Circuit Breaker** for downstream resilience, **Saga** for distributed transaction coordination, **API Gateway** for service aggregation, **CQRS** for read/write separation, and **Outbox Pattern** for reliable event publishing.
- Hands-on **AWS** experience deploying services on EC2 with Auto Scaling, managing data on S3 and RDS, running serverless workloads via Lambda, and monitoring system health through CloudWatch metrics, alarms, and structured log aggregation.
- Strong in unit and integration testing using **JUnit 5**, **Mockito**, **WireMock** for HTTP service simulation, **Testcontainers** for database and Kafka integration tests, and in-memory **H2** for lightweight persistence layer validation.
- Familiar with **CI/CD pipelines** using **Jenkins** and **GitHub Actions**, containerized deployments via **Docker**, and infrastructure-as-code patterns, enabling reliable, repeatable build-test-deploy cycles in Agile environments.

TECHNICAL SKILLS

Languages: Java 8 / 11 / 17, SQL, PL/SQL, Python (scripting)

Backend Frameworks: Spring Boot, Spring MVC, Spring IOC, Spring Security, Spring Data JPA, Spring Batch, Spring AOP, Spring Cloud

Microservices & APIs: Microservices Architecture, RESTful APIs, gRPC, API Gateway, Circuit Breaker, Saga Pattern, CQRS, Outbox Pattern

Messaging & Streaming: Apache Kafka (Producers, Consumers, DLQ, Retry, Partitioning), RabbitMQ, JMS, ActiveMQ

ORM & Persistence: Hibernate (entity lifecycle, lazy loading, N+1 resolution, batch fetch), Spring Data JPA, JDBC

Databases - Relational: PostgreSQL, MySQL, Oracle -- schema design, indexing, query plan analysis, stored procedures, connection pooling (HikariCP)

Databases - NoSQL: MongoDB (aggregation pipelines, indexing, sharding, TTL), Redis (cache-aside, write-through, distributed locking, TTL)

Testing: JUnit 5, Mockito, WireMock, Testcontainers, AssertJ, REST Assured, H2 (in-memory)

Cloud - AWS: EC2, S3, RDS, Lambda, SQS, SNS, CloudWatch, IAM, Secrets Manager, Auto Scaling Groups

DevOps & CI/CD: Docker, Kubernetes, Jenkins, GitHub Actions, Maven, Gradle, SonarQube

Core Java Concepts: Multithreading, Concurrency, ExecutorService, CompletableFuture, Java Streams, Functional Interfaces, Generics, Collections

Design Patterns: Singleton, Factory, Builder, DAO, Proxy, Circuit Breaker, Saga, API Gateway, CQRS, Outbox

Tools: Git, GitHub, JIRA, Confluence, IntelliJ IDEA, Swagger / OpenAPI, Postman, Splunk

PROFESSIONAL EXPERIENCE

Lead Java Backend Developer

February 2024 - Present

Client: **Oracle Corporation** | Dallas, TX

- Architected and built a suite of **Spring Boot microservices** supporting Oracle's enterprise SaaS platform, each independently deployable, horizontally scalable, and aligned with domain-driven boundaries covering tenant management, entitlement workflows, billing events, and usage metering.
- Designed **RESTful APIs** with versioning, HATEOAS-style navigation, paginated list endpoints, and a standardized error response envelope, reducing integration friction for downstream consumers and enabling backward-compatible evolution across API versions.
- Implemented the **API Gateway** pattern using **Spring Cloud Gateway** for dynamic routing, rate limiting per tenant, JWT validation at the edge, and circuit breaking for backend service protection, centralizing cross-cutting concerns away from individual services.
- Applied the **Circuit Breaker** pattern via **Resilience4j** on all outbound service calls, with configurable failure thresholds, half-open state probes, and fallback responses, preventing cascading failures across the service mesh during degraded dependency scenarios.
- Designed and operated **Kafka**-based event pipelines for billing event ingestion, usage record processing, and entitlement state change propagation, handling sustained throughput of millions of events daily with sub-second consumer lag.
- Implemented **Kafka producer idempotency** and exactly-once semantics for billing-critical event flows, ensuring zero duplicate charges even under producer retry and network fault conditions.
- Built consumer group architectures with deliberate partition assignment strategies to parallelize processing across service instances implemented manual offset commit with checkpoint persistence to support resumable, exactly-once-processing guarantees on failure recovery.
- Designed **Dead Letter Queue (DLQ)** workflows for unprocessable events, including automated retry with exponential backoff, alerting on DLQ depth thresholds, and a backfill replay mechanism for bulk reprocessing of corrected records without service redeployment.
- Owned the full persistence layer design for **PostgreSQL**-backed services, covering ER modeling, normalization decisions, index strategy (B-tree on high-cardinality filter columns, partial indexes for soft-delete patterns, composite indexes for multi-column sort/filter queries), and connection pool tuning via **HikariCP**.
- Resolved systemic **N+1 query** problems in Hibernate-managed entities using EntityGraph annotations, batch fetch size configuration, and strategic conversion of JPQL queries to native SQL for complex reporting joins, reducing database round trips on critical read paths by over **60%**.
- Designed and executed **Spring Batch** jobs for end-of-billing-period invoice generation, consumption aggregation, and tenant data export, handling millions of records per run with chunk-oriented processing, skip policies, and restart capability on job failure.
- Integrated **MongoDB** for storing semi-structured tenant configuration, audit trail documents, and usage event metadata designed aggregation pipelines for analytics queries, applied TTL indexes for log retention enforcement, and tuned read preferences for replica set reads.
- Implemented multi-layer **Redis** caching using the cache-aside pattern for tenant context, entitlement lookups, and rate limit counters, achieving over **70% cache hit rate** on hot read paths and reducing P99 API latency by **35%** under sustained load.
- Used **Redis distributed locking** via Redisson to serialize concurrent writes to shared billing state during high-volume period-close operations, eliminating race conditions that previously caused double-charge incidents in pre-production.

- Implemented **Spring Security** with OAuth2 resource server configuration, JWT claim extraction for tenant/user context propagation, and method-level authorization using `@PreAuthorize` annotations, securing multi-tenant APIs against cross-tenant data access.
- Profiled JVM performance using `async-profiler` and heap dump analysis identified off-heap memory pressure from unclosed JDBC connections, thread contention on synchronized cache loaders, and GC pressure from large intermediate collections in stream pipelines, resolving all three in a single performance sprint.
- Established the team's integration testing standard using **Testcontainers**, spinning up real PostgreSQL, MongoDB, and Kafka instances per test suite for deterministic, environment-independent integration coverage without mocking infrastructure away.
- Wrote **Mockito**-based unit tests for all service layer components with **85%+ branch coverage** used **WireMock** to simulate downstream HTTP dependencies with configurable delay and error injection for resilience path testing.
- Mentored **3 mid-level engineers** on Kafka consumer design, Hibernate optimization, and API contract discipline through weekly design reviews and pair-debugging sessions.

***Environment:** Java 17, Spring Boot, Spring MVC, Spring Security (OAuth2 / JWT), Spring Data JPA, Spring Batch, Spring Cloud Gateway, Hibernate, Apache Kafka (Producers, Consumers, DLQ), Resilience4j, PostgreSQL, MongoDB, Redis (Redisson), HikariCP, JUnit 5, Mockito, WireMock, Testcontainers, AssertJ, AWS (EC2, S3, RDS, Lambda, SQS, CloudWatch), Docker, Kubernetes, Maven, Jenkins, GitHub Actions, SonarQube, Swagger / OpenAPI, JIRA, IntelliJ IDEA*

Java Backend Developer

September 2021 - December 2023

Client: Citi Bank | Irving, TX

- Developed backend services for Citi's retail banking platform using **Java 11** and **Spring Boot**, supporting account management, transaction processing, payment initiation, and statement generation workflows for millions of active customers.
- Built **RESTful APIs** for payment and account services consumed by mobile and web channels, enforcing strict input validation, idempotency key handling on payment endpoints, and backward-compatible versioning to support parallel client rollouts.
- Integrated **Apache Kafka** for transaction event streaming between the core banking engine, fraud detection service, and notification platform implemented consumer group rebalancing strategies and lag monitoring to ensure real-time processing SLAs were maintained under peak load.
- Implemented **Spring Security** with JWT and OAuth2, enforcing role-based access control at the API layer and applying field-level encryption for sensitive PII in API responses, meeting internal data governance and PCI-DSS alignment requirements.
- Designed complex **Hibernate** entity models for account and transaction domains applied **optimistic locking** on account balance updates to prevent lost-update anomalies under concurrent modification, using `@Version` fields to detect and surface conflicts cleanly.
- Owned **PostgreSQL** schema design and migration management using **Flyway** designed transaction tables with composite indexes on `account_id` and `transaction_date` for time-range query patterns, and partitioned large transaction tables by month to keep query plans efficient as row counts grew.
- Built **Spring Batch** jobs for monthly statement generation, reconciliation file production, and regulatory reporting data exports, each designed with restartable step execution, record-level skip handling, and audit logging for compliance traceability.
- Implemented **Redis** caching for customer profile lookups, account balance snapshots, and session token storage applied write-through cache updates on balance changes to maintain cache consistency without explicit invalidation complexity.
- Designed and implemented a distributed rate-limiting service using **Redis sorted sets** and sliding window counters to enforce per-customer API quotas, handling **50K+ concurrent sessions** during peak banking hours with no single point of failure.
- Used **CompletableFuture** and Spring's async task executor to parallelize independent downstream calls (credit score fetch, fraud pre-screen, KYC validation) during account opening flows, reducing end-to-end latency from **3.2 seconds to under 900ms**.
- Deployed services on **AWS** infrastructure using EC2 Auto Scaling Groups behind Application Load Balancers managed environment-specific secrets via AWS Secrets Manager and Parameter Store monitored service health through CloudWatch custom metrics and log-based alarms.

- Wrote comprehensive test suites with **JUnit 5** and **Mockito** for service layer coverage used WireMock to stub external payment network responses and Testcontainers for PostgreSQL and Kafka integration tests, maintaining **80%+ coverage** with a fast feedback loop in CI.
- Participated in **Agile** sprints contributing to story refinement, technical design documents, architecture decision records, and post-incident reviews for production issues in critical payment flows.

***Environment:** Java 11, Spring Boot, Spring MVC, Spring Security (JWT / OAuth2), Spring Data JPA, Spring Batch, Hibernate (optimistic locking, Flyway migrations), Apache Kafka, PostgreSQL (partitioning, indexing), Redis (write-through, rate limiting), CompletableFuture, AWS (EC2, S3, RDS, SQS, CloudWatch, Secrets Manager, ALB), Docker, Maven, Jenkins, JUnit 5, Mockito, WireMock, Testcontainers, Swagger / OpenAPI, JIRA*

Junior Java Backend Developer

January 2019 - August 2021

*Client: **Zoho Corporation** | Chennai, India*

- Contributed to backend development of Zoho's CRM and Projects SaaS products using **Java 8** and **Spring MVC**, building server-side features across the contact management, deal pipeline, and task assignment modules.
- Developed and maintained **RESTful API** endpoints for internal and third-party integrations, including Zoho's own analytics and communication products implemented request validation, consistent error handling, and basic API key authentication.
- Built persistence layers using **Hibernate ORM** and **Spring Data JPA**, writing HQL and native SQL queries for reporting features and bulk data export jobs learned query optimization fundamentals through query plan analysis and index experimentation on MySQL.
- Implemented **Spring IOC**-managed service and repository layers with constructor injection, separating business logic cleanly from data access and controller concerns, contributing to a codebase that remained navigable as the team scaled.
- Worked with **MySQL** and **PostgreSQL** for structured product data, and gained hands-on exposure to **MongoDB** for storing flexible user-generated content like custom field definitions and form schemas.
- Applied **Java multithreading** fundamentals using ExecutorService thread pools for background task processing such as email dispatch, report generation, and data sync jobs, to avoid blocking request-handling threads.
- Wrote **JUnit 4** unit tests for service layer methods and learned **Mockito** for dependency mocking during code reviews adopted TDD practices incrementally through team pairing sessions with senior engineers.
- Participated in design discussions around API contract definition, database schema changes, and feature flag rollout strategies, contributing to architectural decisions from an early career stage.
- Assisted in containerizing local development environments with **Docker** and learned the deployment pipeline for Zoho's CI/CD setup using **Jenkins**, gaining foundational DevOps context alongside backend development work.
- Collaborated in **Agile** sprints, attending daily stand-ups, contributing to sprint demos, and maintaining JIRA task tracking across a fast-moving product team delivering biweekly releases.

***Environment:** Java 8, Spring MVC, Spring IOC, Spring Data JPA, Hibernate, MySQL, PostgreSQL, MongoDB, JUnit 4, Mockito, REST APIs, ExecutorService, Docker, Jenkins, Maven, Git, JIRA*